



AKADEMIA GÓRNICZO-HUTNICZA IM. STANISŁAWA STASZICA W KRAKOWIE

WYDZIAŁ INŻYNIERII METALI I INFORMATYKI PRZEMYSŁOWEJ

KATEDRA INFORMATYKI STOSOWANEJ I MODELOWANIA

Projekt dyplomowy

Wykorzystanie technik DevOps w implementacji automatycznego procesu CI/CD do uczenia i wersjonowania modeli sztucznej inteligencji: analiza utrzymania systemów wykorzystujących sztuczną inteligencję w porównaniu do metod manualnych

Using DevOps techniques to implement an automated CI/CD process for training and versioning AI models: an analysis of the maintenance of AI-based systems compared to manual methods

Autor: Rafał Zabłotni
Kierunek studiów: Inżynieria Obliczeniowa
Opiekun pracy: mgr inż. Piotr Hajder

Kraków, 2026

Spis treści

1	Wprowadzenie	3
1.1	Geneza	3
1.2	Założenia	4
2	Podstawy teoretyczne	5
2.1	DevOps	5
2.2	Ciągła Integracja i Ciągłe Dostarczanie (CI/CD)	5
2.3	MLOps	6
2.4	Konteneryzacja - Docker	6
2.5	Kubernetes i k3s	6
2.6	Wersjonowanie modeli - MLflow	7
2.7	Wersjonowanie danych - DVC	7
2.8	Monitoring - Prometheus i Grafana	7
3	Model sztucznej inteligencji	8
3.1	Charakterystyka problemu segmentacji semantycznej	8
3.2	Architektura modelu	8
3.3	Dane treningowe i przygotowanie zbioru danych	9
3.4	Proces uczenia i metryki jakości	10
3.5	Model jako artefakt w procesie CI/CD	10
4	Architektura systemu WIP	11
4.1	Przegląd architektury	11
4.2	Komponenty systemu	11
4.2.1	Warstwa kontroli wersji	11
4.2.2	Warstwa uruchamiania procesów	12
4.2.3	Warstwa przechowywania danych	12
4.2.4	Warstwa monitorowania	13
4.3	Diagram architektury	13
4.4	Przepływ danych w systemie	13
5	Implementacja pipeline CI/CD WIP	14
5.1	Hook pre-push	14
5.2	Walidacja danych	14
5.3	Pipeline treningowy	15
5.4	Bramka jakości	16
5.5	Automatyczne wdrożenie	16
6	Implementacja wdrożenia oraz użycia WIP	18
6.1	Infrastruktura Terraform	18

6.2	Konfiguracja EC2 i k3s	18
6.3	Konfiguracja Helm	19
6.4	Stos monitorowania	19
6.5	Workflow użytkownika	20
6.5.1	Scenariusz 1: Dodanie nowych danych treningowych	20
6.5.2	Scenariusz 2: Predykcja lokalna	20
6.5.3	Scenariusz 3: Ręczne uruchomienie treningu	20
7	Eksperymenty i analiza wyników WIP	21
7.1	Metodologia porównania	21
7.2	Czasy wykonania scenariuszy	21
7.3	Metryki modelu	21
7.4	Metryki infrastruktury	22
7.4.1	Zużycie zasobów podczas treningu	22
7.4.2	Zużycie zasobów podczas serwowania	22
7.4.3	Koszty infrastruktury AWS	22
7.5	Ocena jakościowa	22
7.5.1	Łatwość wdrożenia	22
7.5.2	Stabilność systemu	22
7.5.3	Skalowalność	22
8	Podsumowanie i wnioski WIP	23
8.1	Realizacja celów pracy	23
8.2	Wnioski	23
8.3	Ograniczenia	23
8.4	Kierunki dalszego rozwoju	24
	Literatura	25

1 Wprowadzenie

1.1 Geneza

Pomysł na realizację niniejszego projektu zrodził się podczas wykonywania codziennych obowiązków zawodowych. W trakcie pracy nad jednym z projektów zaobserwowano interesującą tendencję związaną z procesem tworzenia testów modułowych. Znaczna część kodu testowego okazywała się redundantna, a wiele mechanizmów było wielokrotnie powielanych poprzez kopiowanie oraz wprowadzanie drobnych modyfikacji do istniejących fragmentów kodu.

Naturalnym rozwiązaniem tego problemu mogłoby wydawać się wydzielanie powtarzalnych elementów do funkcji pomocniczych lub bibliotek testowych oraz utrzymywanie ich w uporządkowanej i współdzielonej formie. Podejście to jednak wiąże się z istotnymi trudnościami, zarówno natury organizacyjnej, jak i technicznej. W praktyce często prowadzi ono do nadmiernego rozrostu bibliotek pomocniczych, niejednoznacznych interfejsów oraz problemów z ich długoterminowym utrzymaniem.

W szczególności pojawiają się następujące problemy:

1. Jak określić moment, w którym dana funkcjonalność jest wystarczająco generyczna, aby mogła zostać wydzielona do wspólnej biblioteki?
2. Jak zminimalizować ryzyko tworzenia funkcji lokalnie w plikach testowych, które z czasem zaczynają być wykorzystywane przez inne testy, prowadząc do niejawnych zależności pomiędzy teoretycznie niezależnymi modułami?
3. Jak zapewnić spójny sposób tworzenia oraz utrzymania takich funkcji w projektach rozwijanych przez dziesiątki, a niekiedy setki programistów, o różnym poziomie doświadczenia i odmiennych nawykach pracy?

Jednocześnie, jak powszechnie wiadomo, modele sztucznej inteligencji uczą się na podstawie zbiorów danych treningowych, a następnie wykorzystują wyuczone wzorce do rozwiązywania nowych problemów. Obserwacja ta stała się punktem wyjścia do rozważenia możliwości zastosowania modeli AI w obszarze automatycznego generowania wspólnych, powtarzalnych fragmentów kodu testowego na podstawie już istniejących przykładów.

Jednakże, aby taki system mógł być skutecznie wykorzystany w praktyce, konieczne jest zapewnienie odpowiedniego procesu ciągłej integracji i dostarczania (CI/CD), który umożliwi automatyczne trenowanie, ocenę jakości oraz wdrażanie modeli AI w sposób powtarzalny i kontrolowany, lub oddelegowanie pracownika do utrzymania takiego procesu w sposób manualny.

W ten sposób pojawiają dwa kluczowe pytania, które stanowią oś projektu:

1. Czy możliwe i opłacalne jest stworzenie modelu SI który będzie obsługiwał automatyczne generowanie fragmentów kodu testowego?
2. Czy bardziej opłacalne jest oddelegowanie jednego pracownika do ręcznego utrzymywania i rozwijania tego oprogramowania, czy też lepszym rozwiązaniem jest automatyzacja tego procesu z wykorzystaniem narzędzi DevOps oraz pipeline'ów CI/CD?

1.2 Założenia

Ze względu na złożoność oryginalnego problemu generowania kodu testowego oraz ograniczony czas realizacji projektu, zdecydowano się na zastosowanie problemu zastępczego, który pozwala w pełni zweryfikować hipotezę bez konieczności budowania zaawansowanego modelu językowego. Jako taki substytut wybrano segmentację semantyczną obrazów wodomierzy z wykorzystaniem architektury U-Net. Wybór ten jest celowy: model segmentacji obrazów posiada wyraźnie zdefiniowane metryki jakości (współczynnik Dice, IoU), umiarkowane wymagania obliczeniowe umożliwiające trening w ramach dostępnego budżetu, a dostępność wstępnie wytrenowanego modelu bazowego pozwala na obiektywne porównanie kolejnych wersji. Dla uproszczenia przyjęto, że model SI dostarczany jest przez zewnętrzną firmę, natomiast zakres pracy obejmuje jego dalsze uczenie, ocenę jakości oraz wdrażanie. Takie założenie odzwierciedla realny scenariusz w środowisku produkcyjnym, gdzie zespół DevOps otrzymuje gotowy model i odpowiada za zarządzanie jego cyklem życia.

W tak zdefiniowanym kontekście drugie pytanie sformułowane w poprzedniej sekcji pozostaje w pełni aktualne: czy bardziej opłacalne jest oddelegowanie pracownika do ręcznego utrzymywania i rozwijania modelu, czy też lepszym rozwiązaniem jest automatyzacja tego procesu z wykorzystaniem narzędzi DevOps i pipeline'ów CI/CD? W podejściu manualnym proces ponownego uczenia i wdrażania modelu wymaga ręcznego przygotowania danych, uruchamiania treningu, walidacji wyników oraz publikacji nowej wersji. Takie podejście, choć proste koncepcyjnie, wiąże się z ryzykiem błędów ludzkich, trudnościami w zachowaniu powtarzalności eksperymentów oraz ograniczoną skalowalnością. Automatyzacja natomiast umożliwia jednoznaczne powiązanie wersji modelu z wersją danych treningowych, wprowadzenie obiektywnej bramki jakości i eliminację ręcznych kroków, które mogą prowadzić do niespójności.

W ramach niniejszej pracy opracowano kompletny, zautomatyzowany system CI/CD dla modeli sztucznej inteligencji. System obejmuje wersjonowanie danych treningowych (DVC), śledzenie eksperymentów i rejestr modeli (MLflow), automatyczny pipeline treningowy uruchamiany przez GitHub Actions, bramkę jakości porównującą nowy model z aktualną wersją produkcyjną, a także wdrożenie modelu w środowisku chmurowym (k3s na EC2) z możliwością użycia aplikacji oraz monitoringiem (Prometheus, Grafana). Całość zaprojektowano tak, aby dodanie nowych danych treningowych przez użytkownika wymagało jedynie wykonania `git push`, a dalszy proces odbywał się bez jego udziału. Tak zdefiniowany zakres pozwala na bezpośrednie porównanie kosztów i nakładu pracy podejścia manualnego i automatycznego, co stanowi główny cel analityczny pracy.

2 Podstawy teoretyczne

2.1 DevOps

DevOps to zbiór praktyk, kultury organizacyjnej i narzędzi, których celem jest skrócenie cyklu życia oprogramowania przy jednoczesnym zapewnieniu wysokiej jakości dostarczanych produktów. Termin pochodzi od połączenia słów *Development* i *Operations*, co odzwierciedla dążenie do przełamania tradycyjnej bariery między zespołami deweloperskimi a operacyjnymi.

Kluczowym elementem filozofii DevOps jest pętla ciągłego doskonalenia, obejmująca planowanie, kodowanie, budowanie, testowanie, wdrażanie, uruchamianie oraz monitorowanie. Każdy obrót pętli dostarcza informacje zwrotne, które pozwalają szybciej reagować na błędy i wymagania użytkowników. W kontekście niniejszej pracy DevOps stanowi ramy koncepcyjne, w których osadzony jest cały projekt - automatyzacja procesu trenowania i wdrażania modelu SI jest bezpośrednią aplikacją tych zasad [12].

2.2 Ciągła Integracja i Ciągłe Dostarczanie (CI/CD)

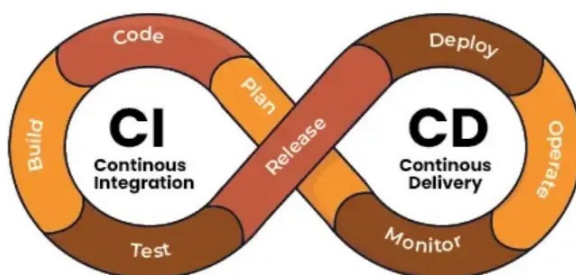
Ciągła Integracja (ang. *Continuous Integration*, CI) polega na automatycznym scalaniu i weryfikacji zmian kodu wielokrotnie w ciągu dnia. Każda zmiana uruchamia zdefiniowany zestaw testów, co pozwala wykryć błędy integracyjne natychmiast po ich wprowadzeniu, zamiast odkrywać je dopiero na etapie wydania.

Ciągłe Dostarczanie (ang. *Continuous Delivery*) rozszerza CI o automatyczne przygotowanie artefaktów gotowych do wdrożenia. Ciągłe Wdrażanie (ang. *Continuous Deployment*) idzie krok dalej - każda zmiana spełniająca kryteria jakości jest wdrażana do produkcji.

Typowy pipeline CI/CD składa się z następujących etapów:

1. weryfikacja kodu (linting, testy jednostkowe),
2. budowanie artefaktów (np. obrazu Docker),
3. testy integracyjne i akceptacyjne,
4. wdrożenie na środowisko docelowe,
5. weryfikacja po wdrożeniu (smoke tests).

W niniejszym projekcie pipeline CI/CD realizowany jest przez GitHub Actions i obejmuje dodatkowe etapy specyficzne dla ML: walidację danych, trening modelu oraz bramkę jakości.



Rys. 1 Przykład flow CI/CD [14].

2.3 MLOps

MLOps (ang. *Machine Learning Operations*) to dyscyplina stosująca zasady DevOps do specyfikacji cyklu życia modeli uczenia maszynowego. W odróżnieniu od klasycznego oprogramowania, gdzie artefaktem jest kod, w ML artefaktem jest model - wynik trenowania na konkretnym zbiorze danych. To wprowadza kilka dodatkowych wyzwań:

- dane treningowe zmieniają się niezależnie od kodu,
- wyniki trenowania są niedeterministyczne,
- jakość modelu musi być mierzona obiektywnie przed każdym wdrożeniem,
- modele degradują się w czasie (tzw. *data drift*).

Cykl życia modelu ML obejmuje zbieranie danych, przygotowanie danych, eksperymentowanie, trening, ewaluację, wdrożenie i monitoring. MLOps automatyzuje ten cykl, integrując go z narzędziami kontroli wersji, CI/CD oraz rejestrami modeli. W niniejszym projekcie MLOps realizowany jest przez połączenie GitHub Actions (automatyzacja), DVC (wersjonowanie danych) oraz MLflow (śledzenie eksperymentów i rejestr modeli).

2.4 Konteneryzacja - Docker

Docker to platforma do tworzenia, dystrybucji i uruchamiania aplikacji w kontenerach [13]. Kontener to izolowane środowisko uruchomieniowe zawierające aplikację wraz ze wszystkimi jej zależnościami: bibliotekami, interpreterem, zmiennymi środowiskowymi. Dzięki temu aplikacja działa identycznie na maszynie dewelopera, serwerze testowym i produkcji, eliminując problem „u mnie działa”.

Kluczowe pojęcia:

- **Obraz (image)** - niezmienny szablon opisujący środowisko, plik *Dockerfile*,
- **Kontener** - działająca instancja obrazu; izolowana od systemu hosta i innych kontenerów,
- **Rejestr (registry)** - repozytorium obrazów (np. Docker Hub, Amazon ECR).

W projekcie Docker służy do pakowania API modelu oraz do zapewnienia powtarzalności środowiska treningowego. Obraz z modelem jest budowany automatycznie w pipeline po zatwierdzeniu nowej wersji modelu.

2.5 Kubernetes i k3s

Kubernetes (K8s) to system do orkiestracji kontenerów - automatyzuje ich wdrażanie, skalowanie i zarządzanie [2]. Zamiast uruchamiać kontenery ręcznie na konkretnych maszynach, Kubernetes przyjmuje deklaracyjny opis pożądanego stanu (ile replik, jakie zasoby, jak reagować na awarie) i utrzymuje ten stan niezależnie od zdarzeń infrastrukturalnych.

k3s to uproszczona dystrybucja Kubernetes stworzona przez Rancher Labs. Zmniejsza wymagania sprzętowe (minimalna instalacja działa na instancji z 2 GB RAM) i upraszcza instalację do pojedynczego polecenia. Jest przeznaczona dla środowisk brzegowych, IoT oraz małych klastrów. W niniejszym projekcie k3s zainstalowany na pojedynczej instancji EC2 zastępuje zarządzany klaster EKS, co pozwala ograniczyć koszty do minimum.

2.6 Wersjonowanie modeli - MLflow

MLflow to platforma open-source do zarządzania cyklem życia modeli ML. Składa się z czterech głównych komponentów:

- **Tracking** - rejestrowanie parametrów, metryk i artefaktów każdego uruchomienia eksperymentu,
- **Projects** - standaryzacja i reprodukowalność kodu ML,
- **Models** - zunifikowany format pakowania modeli,
- **Model Registry** - centralny rejestr wersji modeli z etapami cyklu życia (Staging, Production, Archived).

Model Registry jest kluczowym elementem w kontekście bramki jakości. Po każdym treningu nowa wersja modelu jest rejestrowana w MLflow. Bramka jakości pobiera metryki modelu aktualnie w etapie Production i porównuje z nowym wynikiem. Tylko model z lepszymi metrykami zostaje awansowany do Production [15].

2.7 Wersjonowanie danych - DVC

DVC (ang. *Data Version Control*) to narzędzie umożliwiające wersjonowanie dużych plików binarnych (obrazów, wag modeli, zbiorów danych) w sposób spójny z Git. Zamiast przechowywać dane w repozytorium Git (co jest niepraktyczne przy dużych plikach), DVC przechowuje jedynie lekkie pliki metadanych (`.dvc`), a rzeczywiste dane trafiają do zewnętrznego magazynu - w tym projekcie do Amazon S3 [8].

Zasada działania jest analogiczna do Git: każda wersja danych ma swój hash, można cofać się do poprzednich wersji poleceniem `dvc checkout`, a polecenie `dvc pull` pobiera dane odpowiadające bieżącemu stanowi repozytorium. Dzięki temu możliwe jest odtworzenie dokładnego eksperymentu: tej samej wersji kodu, danych i konfiguracji.

2.8 Monitoring - Prometheus i Grafana

Prometheus to system monitorowania i alertowania oparty na modelu *pull* [16]. Regularnie pobiera (scrape) metryki z endpointów HTTP udostępnianych przez monitorowane usługi i przechowuje je w wbudowanej bazie szeregów czasowych. Metryki opisywane są etykietami (labels), co pozwala na elastyczne filtrowanie i agregację.

Grafana to platforma wizualizacji danych, która integruje się z wieloma źródłami, w tym z Prometheus. Pozwala tworzyć interaktywne dashboardy z wykresami, wskaźnikami i alertami. W projekcie Grafana prezentuje metryki działającego API modelu: liczbę predykcji, czasy odpowiedzi, zużycie zasobów i liczbę błędów. Monitoring działa jako opcjonalny stos uruchamiany na tym samym klastrze k3s co API modelu [11].

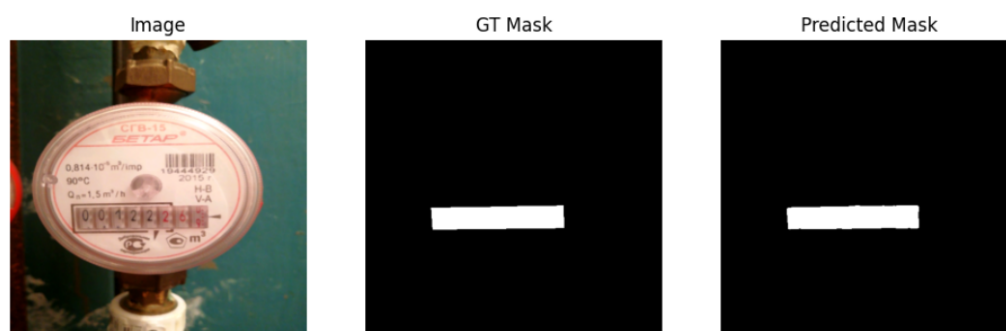
3 Model sztucznej inteligencji

3.1 Charakterystyka problemu segmentacji semantycznej

Model sztucznej inteligencji stanowiący podstawę niniejszego projektu został zaprojektowany do rozwiązywania zadania **segmentacji semantycznej obrazów wodomierzy**. Celem segmentacji jest przypisanie każdemu pikselowi obrazu etykiety określającej jego przynależność do jednej z wyróżnionych klas, w szczególności obszaru licznika, tarczy oraz tła.

Problem ten ma istotne znaczenie praktyczne w kontekście automatyzacji odczytu wskazań wodomierzy, gdzie poprawne wydzielenie obszaru licznika stanowi warunek konieczny dla dalszych etapów przetwarzania obrazu, takich jak rozpoznawanie cyfr lub analiza wskazań mechanicznych. W przeciwieństwie do klasycznej klasyfikacji obrazów, segmentacja semantyczna wymaga zachowania informacji przestrzennej oraz precyzyjnego odwzorowania granic obiektów, co znacząco zwiększa złożoność problemu.

Dodatkowym wyzwaniem są zmienne warunki akwizycji danych, obejmujące różnice w oświetleniu, kącie wykonania zdjęcia, jakości obrazu oraz stopniu zabrudzenia lub zużycia urządzeń pomiarowych. Czynniki te powodują, że klasyczne algorytmy przetwarzania obrazu okazują się niewystarczające, co uzasadnia zastosowanie metod opartych na uczeniu głębokim.



Rys. 2 Przykład predykcji modelu segmentacji semantycznej

3.2 Architektura modelu

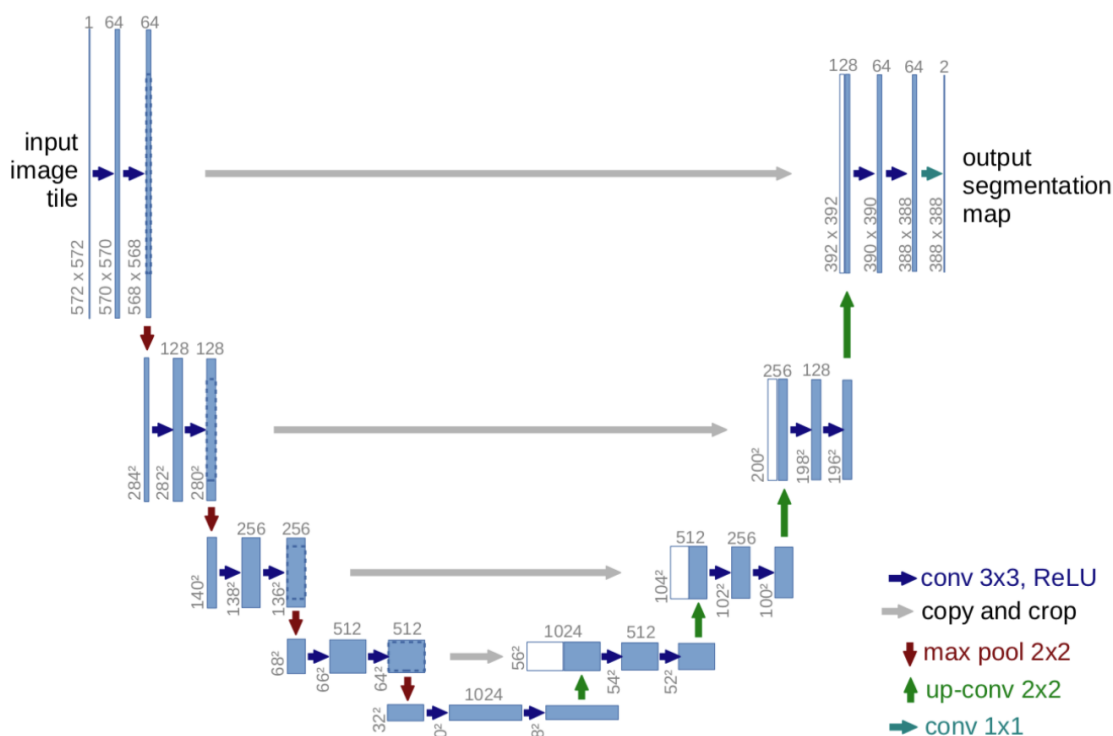
W projekcie wykorzystano konwolucyjną sieć neuronową opartą na architekturze **U-Net**, powszechnie stosowaną w zadaniach segmentacji semantycznej obrazów. Architektura ta charakteryzuje się symetryczną budową typu encoder-decoder, umożliwiającą jednocześnie wydobycie cech semantycznych oraz zachowanie informacji przestrzennej.

Część kodująca (encoder) odpowiada za stopniowe przekształcanie obrazu wejściowego do reprezentacji o coraz wyższym poziomie abstrakcji poprzez sekwencję operacji konwencji oraz redukcji rozdzielczości. Część dekodująca (decoder) realizuje proces rekonstrukcji obrazu wyjściowego, przywracając jego pierwotną rozdzielczość i generując maskę segmentacyjną.

Kluczowym elementem architektury są tzw. *skip connections*, umożliwiające bezpośrednie przekazywanie map cech pomiędzy odpowiadającymi sobie poziomami enkodera i dekodera.

Mechanizm ten pozwala ograniczyć utratę informacji lokalnej oraz poprawić jakość segmentacji, szczególnie w obszarach granicznych pomiędzy klasami.

Zastosowana implementacja architektury U-Net została dostosowana do specyfiki problemu, w tym rozdzielczości obrazów wejściowych oraz liczby klas segmentacyjnych. Model generuje maskę wyjściową o tej samej rozdzielczości co obraz wejściowy, co ułatwia dalsze etapy przetwarzania oraz ocenę jakości predykcji.



Rys. 3 Schemat architektury U-Net [4]

3.3 Dane treningowe i przygotowanie zbioru danych

Model został wytrenowany na zbiorze danych składającym się z obrazów wodomierzy oraz odpowiadających im masek. Każda próbka zawiera obraz wejściowy w postaci fotografii oraz maskę określającą przynależność poszczególnych pikseli do zdefiniowanych klas.

Proces przygotowania danych obejmował normalizację wartości pikseli, standaryzację rozdzielczości obrazów oraz podział zbioru danych na część treningową, walidacyjną i testową. W celu zwiększenia odporności modelu na zmienność danych wejściowych zastosowano techniki augmentacji danych, takie jak losowe obroty, skalowanie, przesunięcia oraz modyfikacje jasności i kontrastu.

Zastosowanie augmentacji pozwala ograniczyć ryzyko przeuczenia modelu oraz poprawić jego zdolność do generalizacji na dane niewidziane w procesie uczenia. Istotnym aspektem projektu jest fakt, że zbiór danych oraz jego kolejne wersje podlegają wersjonowaniu, co umożliwia jednoznaczne powiązanie konkretnej wersji modelu z użytymi danymi treningowymi.

Zbiór danych wykorzystany w projekcie pochodzi z platformy Kaggle [9].

3.4 Proces uczenia i metryki jakości

Proces uczenia modelu realizowany jest w sposób nadzorowany, z wykorzystaniem funkcji straty dostosowanej do problemu segmentacji semantycznej. Jako główne metryki oceny jakości modelu zastosowano **współczynnik Dice** oraz **Intersection over Union (IoU)**, które są standardowymi miarami stosowanymi w zadaniach segmentacyjnych.

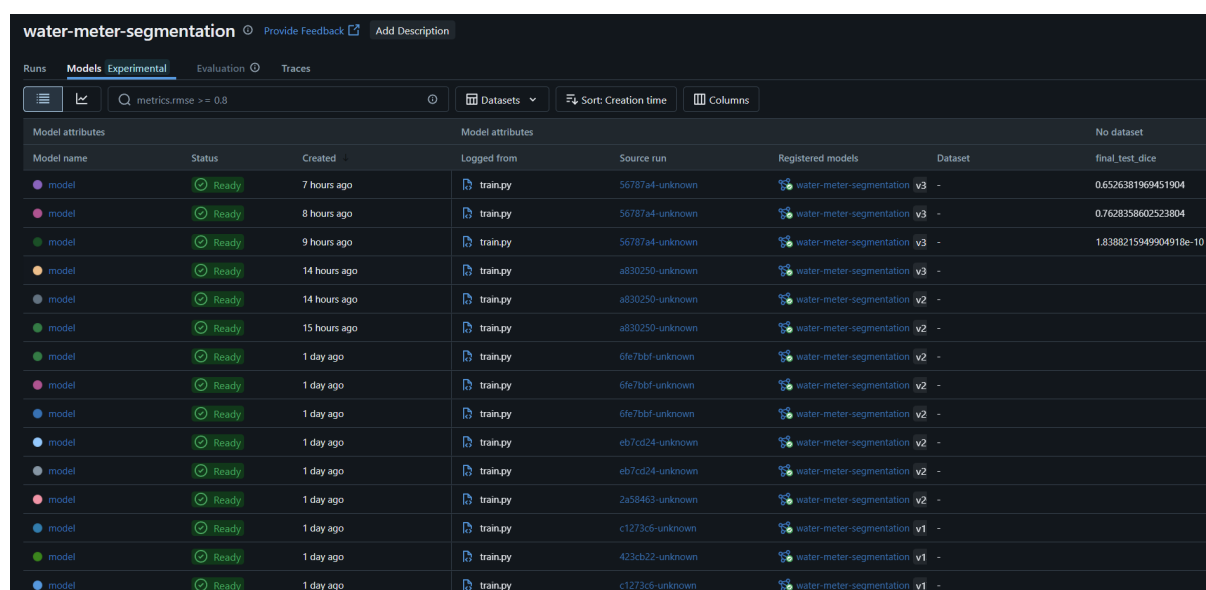
Współczynnik Dice umożliwia ocenę stopnia pokrycia obszaru przewidywanego przez model z obszarem referencyjnym, natomiast IoU mierzy stosunek części wspólnej obu obszarów do ich sumy. Zastosowanie obu metryk pozwala na bardziej kompleksową ocenę jakości predykcji, szczególnie w przypadku niebalansowanych klas.

Proces uczenia realizowany jest iteracyjnie, z wykorzystaniem zbioru walidacyjnego do monitorowania jakości modelu oraz ograniczania zjawiska przeuczenia. Najlepsza wersja modelu, zgodnie z przyjętymi kryteriami jakości, zapisywana jest jako artefakt i przekazywana do dalszych etapów zautomatyzowanego pipeline'u CI/CD.

3.5 Model jako artefakt w procesie CI/CD

W kontekście niniejszej pracy model sztucznej inteligencji traktowany jest jako pełnoprawny artefakt inżynierski, podlegający wersjonowaniu, testowaniu oraz kontrolowanemu wdrażaniu. Każda wersja modelu jest jednoznacznie powiązana z wersją kodu źródłowego, zestawem danych treningowych, konfiguracją hiperparametrów oraz uzyskanymi metrykami jakości.

Takie podejście umożliwia pełną reprodukowalność wyników oraz stanowi podstawę do dalszej analizy porównawczej pomiędzy podejściem manualnym a zautomatyzowanym procesem CI/CD. Szczegółowa analiza wpływu automatyzacji na jakość, stabilność oraz koszty utrzymania modeli sztucznej inteligencji zostanie przedstawiona w kolejnych rozdziałach pracy.



water-meter-segmentation							
Runs Models Experimental Evaluation Traces							
metrics.rmse >= 0.8							
Datasets Sort: Creation time Columns							
Model attributes				Model attributes			
Model name	Status	Created	Logged from	Source run	Registered models	Dataset	No dataset
model	Ready	7 hours ago	train.py	56787a4-unknown	water-meter-segmentation v3	-	0.6526381969451904
model	Ready	8 hours ago	train.py	56787a4-unknown	water-meter-segmentation v3	-	0.7628358602523804
model	Ready	9 hours ago	train.py	56787a4-unknown	water-meter-segmentation v3	-	1.8388215949904918e-10
model	Ready	14 hours ago	train.py	a830250-unknown	water-meter-segmentation v3	-	
model	Ready	14 hours ago	train.py	a830250-unknown	water-meter-segmentation v2	-	
model	Ready	15 hours ago	train.py	a830250-unknown	water-meter-segmentation v2	-	
model	Ready	1 day ago	train.py	6fe7bbf-unknown	water-meter-segmentation v2	-	
model	Ready	1 day ago	train.py	6fe7bbf-unknown	water-meter-segmentation v2	-	
model	Ready	1 day ago	train.py	6fe7bbf-unknown	water-meter-segmentation v2	-	
model	Ready	1 day ago	train.py	eb7cd24-unknown	water-meter-segmentation v2	-	
model	Ready	1 day ago	train.py	eb7cd24-unknown	water-meter-segmentation v2	-	
model	Ready	1 day ago	train.py	2a58463-unknown	water-meter-segmentation v2	-	
model	Ready	1 day ago	train.py	c1273c6-unknown	water-meter-segmentation v1	-	
model	Ready	1 day ago	train.py	423cb22-unknown	water-meter-segmentation v1	-	
model	Ready	1 day ago	train.py	c1273c6-unknown	water-meter-segmentation v1	-	

Rys. 4 Wersjonowanie modeli w rejestrze MLflow

4 Architektura systemu WIP

4.1 Przegląd architektury

System został zaprojektowany jako zautomatyzowany pipeline CI/CD dla modeli uczenia maszynowego. Obejmuje cały cykl życia modelu: od dodania danych treningowych, przez walidację danych i trening, aż po wybór najlepszego modelu i wdrożenie.

Architektura opiera się na trzech głównych obszarach:

- kontrola wersji kodu i danych,
- uruchamianie zadań i wdrożeń w kontenerach,
- rejestrowanie eksperymentów oraz wersji modeli.

Punktem centralnym jest repozytorium Git. To w nim przechowywany jest kod, konfiguracja oraz metadane danych. Każdy push lub merge może uruchomić automatyczne procesy w GitHub Actions [5]. Dzięki temu użytkownik nie musi ręcznie wykonywać większości kroków operacyjnych.

Część obliczeniowa działa w chmurze AWS. Zasoby są uruchamiane tylko wtedy, gdy są potrzebne, na przykład na czas treningu lub pobrania modelu. Po zakończeniu pracy instancja jest zatrzymywana, aby ograniczyć koszty. Szczegółowe uzasadnienie takich decyzji znajduje się we wprowadzeniu w sekcji 1.

W projekcie wykorzystano model U-Net do segmentacji semantycznej obrazów wodomierzy. Metryki Dice oraz IoU są używane w bramce jakości, która decyduje, czy nowa wersja modelu może zostać uznana za lepszą od obecnej.

4.2 Komponenty systemu

4.2.1 Warstwa kontroli wersji

Git odpowiada za wersjonowanie kodu, konfiguracji oraz metadanych związanych z danymi treningowymi. Repozytorium można podzielić na trzy logiczne części.

Kod modelu i skrypty treningowe

Ta część zawiera implementację modelu U-Net w PyTorch oraz skrypty odpowiedzialne za przygotowanie danych, trening i walidację. Ponieważ kod jest wersjonowany, można odtworzyć, w jakich warunkach trenowano konkretną wersję modelu.

Dane treningowe

Domyślną metodą zarządzania danymi treningowymi w systemie jest DVC [8]. Repozytorium Git przechowuje wtedy jedynie metadane i pliki śledzące (na przykład `.dvc` oraz `dvc.lock`), natomiast właściwe obrazy i maski znajdują się w Amazon S3. Takie podejście skaluje się znacznie lepiej niż trzymanie dużych plików w Git i pozwala na wersjonowanie danych w sposób spójny z wersjonowaniem kodu.

W wariancie proof of concept dopuszczono również możliwość przechowywania małych zbiorów danych bezpośrednio w repozytorium, aby uprościć szybkie testy. Należy jednak zaznaczyć, że użycie DVC wiąże się z dodatkowym krokiem `dvc pull`, który w praktyce może zauważalnie wydłużyć czas działania pipeline, szczególnie przy większych zbiorach danych.

Infrastruktura jako kod

Definicja infrastruktury jest utrzymywana w kodzie, głównie jako moduły Terraform [1] oraz konfiguracje Helm [6]. Dzięki temu środowisko można odtworzyć w sposób powtarzalny, a zmiany infrastruktury są widoczne w historii repozytorium.

4.2.2 Warstwa uruchamiania procesów

GitHub Actions

GitHub Actions pełni rolę głównego mechanizmu uruchamiania procesów CI/CD. W projekcie wykorzystano kilka workflow, z których najważniejsze realizują następujące zadania:

- walidacja danych treningowych,
- uruchomienie treningu modelu i zebranie wyników,
- uruchamianie i zatrzymywanie instancji EC2,
- testy jakości kodu i testy jednostkowe,
- obsługa procesu dodania danych i utworzenia pull requesta.

Instancja EC2 i k3s

Zamiast zarządzanego klastra Kubernetes zastosowano k3s [17] uruchomiony na pojedynczej instancji EC2. To podejście jest wystarczające dla projektu proof of concept i pozwala znacząco ograniczyć koszty. Na instancji działa klaster k3s oraz usługi potrzebne do treningu i wdrożeń.

MLflow

MLflow [18] odpowiada za dwie rzeczy. Po pierwsze zapisuje przebieg eksperymentów, czyli metryki, parametry oraz artefakty treningu. Po drugie przechowuje wersje modeli i ich status, na przykład Production lub Archived. W tym projekcie metadane są przechowywane w bazie lokalnej na instancji EC2, a artefakty trafiają do S3.

4.2.3 Warstwa przechowywania danych

Amazon S3

S3 jest używane jako trwałe miejsce przechowywania. W systemie trafiają tam głównie artefakty z MLflow, takie jak wagi modeli i logi. Opcjonalnie S3 może też być wykorzystywane do przechowywania danych treningowych, jeśli używany jest DVC.

Amazon ECR

ECR służy jako rejestr obrazów Docker. Po zaakceptowaniu modelu przez bramkę jakości obraz kontenera jest budowany i wysyłany do ECR, a następnie wdrażany na klaster k3s.

Warto zaznaczyć, że jeśli dane są przechowywane w DVC, etap przygotowania środowiska obejmuje dodatkowe pobranie danych z S3, co jest jednym z głównych źródeł wydłużenia czasu całego workflow treningowego.

4.2.4 Warstwa monitorowania

System zawiera komponenty monitoringu oparte na Prometheus [16] oraz Grafana. Prometheus zbiera metryki z endpointu `/metrics` udostępnionego przez API modelu, a Grafana prezentuje je w postaci dashboardów. Szczegóły konfiguracji stosu monitorowania opisane są w rozdziale 6.

4.3 Diagram architektury

[Do uzupełnienia: diagram C4 architecture-overview.png]

Rys. 5 Diagram kontekstowy C4 przedstawiający główne komponenty systemu oraz ich interakcje

Rysunek 5 przedstawia kontekst systemu w notacji C4. Użytkownik pracuje z repozytorium Git, a automatyzacja realizowana jest przez GitHub Actions. Workflow zarządzają zasobami AWS, uruchamiają instancję EC2 i wykonują zadania treningowe oraz wdrożeniowe. Na EC2 działa klaster k3s, w którym uruchomione są MLflow, API modelu oraz narzędzia monitoringu. Artefakty modelu są przechowywane w S3, a obrazy kontenerów w ECR.

4.4 Przepływ danych w systemie

[Do uzupełnienia: diagram przepływu danych data-flow.png]

Rys. 6 Diagram przepływu danych treningowych przez system

Rysunek 6 pokazuje drogę danych od lokalnego repozytorium do artefaktów produkcyjnych. Dane są wersjonowane przez DVC, a następnie pobierane przez runner GitHub Actions. Skrypty przygotowują podział danych, trening generuje wagi modelu, a wynik jest rejestrowany w MLflow. Model Production może zostać pobrany lokalnie i użyty do predykcji.

5 Implementacja pipeline CI/CD WIP

Rozdział opisuje implementację zautomatyzowanego pipeline CI/CD dla modeli uczenia maszynowego. Prezentowane rozwiązanie realizuje przepływ: dodanie danych treningowych przez użytkownika wyzwala sekwencję kroków — walidację, trening, ocenę jakości i wdrożenie — bez konieczności ręcznej interwencji.

5.1 Hook pre-push

Pierwszym elementem pipeline jest hook Git pre-push [3], instalowany lokalnie na maszynie użytkownika przez skrypt `devops/scripts/install-git-hooks.sh`. Hook uruchamia się automatycznie przed każdym `git push` i sprawdza, czy w zestawie zmian znajdują się pliki w katalogach `WMS/data/training/images/` lub `WMS/data/training/masks/`.

Jeśli zmiany dotyczą danych treningowych, hook:

1. tworzy nową gałąź o nazwie `data/TIMESTAMP` (np. `data/20260215-143022`),
2. commituje do niej wyłącznie nowe pliki z katalogów danych,
3. przekierowuje push na tę gałąź zamiast na `main`.

Całość wykonuje się w mniej niż 10 sekund, ponieważ hook nie wymaga połączenia z AWS ani lokalnych poświadczeń chmurowych. Merge danych z historycznym zbiorem w S3 odbywa się po stronie GitHub Actions.

5.2 Walidacja danych

Po wypchnięciu gałęzi danych uruchamia się pierwszy job workflow `training-data-pipeline.yaml` [5]: `merge-and-validate`. Składa się on z dwóch faz.

Faza scalania

Job pobiera istniejące dane z Amazon S3 (za pomocą poświadczeń przechowywanych w sekretach GitHub), scala je z nowo dostarczonymi plikami i aktualizuje manifesty DVC. Scalanie jest idempotentne — pliki o identycznej nazwie i sumie kontrolnej nie są duplikowane.

Faza walidacji

Skrypt `validate_data.py` weryfikuje scalony zbiór danych pod kątem czterech kryteriów:

- **kompletność par** — dla każdego obrazu w katalogu `images/` musi istnieć maska o tej samej nazwie w `masks/` (porównanie po *stem* nazwy pliku),
- **rozdzielczość** — każdy plik obrazu i maski musi mieć rozdzielczość dokładnie 512×512 px,
- **binarność masek** — maski mogą zawierać wyłącznie wartości 0 i 255 (brak półprzezroczystości),
- **format pliku** — akceptowane są formaty `.jpg` oraz `.png`.

Wynik walidacji jest publikowany jako komentarz w pull requeście. W przypadku błędów komentarz zawiera listę plików niezgodnych z wymaganiami, a dalsze jobe są pomijane (warunkowanie przez `needs` i `if: needs.merge-and-validate.outputs.valid == 'true'`).

```
# Fragment skryptu walidacji
for img_stem in image_stems:
    if img_stem not in mask_stems:
        errors.append(f"Brak maski dla: {img_stem}")
```

5.3 Pipeline treningowy

Główny workflow `training-data-pipeline.yaml` składa się z sześciu jobów wykonywanych sekwencyjnie z wyjątkiem `stop-infra`, który uruchamia się zawsze niezależnie od wyników poprzednich kroków.

Rys. 7 Sekwencja jobów w workflow `training-data-pipeline.yaml`

Job 1: merge-and-validate

Opisany w poprzedniej sekcji. Pobiera dane z S3, scala, waliduje i aktualizuje DVC.

Job 2: start-infra

Uruchamia instancję EC2 za pomocą AWS CLI (`aws ec2 start-instances`). Następnie czeka na pojawienie się GitHub Actions runnera na tej instancji, odpytując API GitHub co 30 sekund przez maksymalnie 10 minut.

Job 3: train

Wykonuje się na self-hosted runnerze zainstalowanym na instancji EC2. Pobiera dane z S3, uruchamia skrypt `train.py`, który trenuje model U-Net przez zdefiniowaną liczbę epok i loguje metryki do MLflow. Po zakończeniu treningu skrypt `quality_gate.py` porównuje wyniki z modelem Production.

Job 4: stop-infra

Zatrzymuje instancję EC2 niezależnie od wyniku treningu (`if: always()`). Zapobiega to przypadkowemu pozostawieniu instancji uruchomionej i naliczaniu kosztów.

Job 5: create-pr

Uruchamia się tylko jeśli trening zakończył się sukcesem i model spełnił kryteria bramki jakości (`if: needs.train.outputs.improved == 'true'`). Tworzy pull request z gałęzi danych do `main` z opisem zawierającym metryki nowego modelu i poprzedniego baseline.

Job 6: auto-merge

Włącza automatyczne scalanie pull requesta po pozytywnym przejściu statusowych sprawdzeń wymaganych przez reguły ochrony gałęzi.

5.4 Bramka jakości

Bramka jakości jest mechanizmem decydującym, czy nowy model jest wystarczająco dobry, żeby trafić do produkcji. Implementacja jest w skrypcie `quality_gate.py` i korzysta z API MLflow [18].

Pobieranie baseline

Skrypt łączy się z MLflow przez `MlflowClient` i pobiera metryki aktualnego modelu Production:

```
client = MlflowClient(tracking_uri=mlflow_uri)
versions = client.get_latest_versions(
    "water-meter-segmentation",
    stages=["Production"]
)
if versions:
    baseline_run = client.get_run(versions[0].run_id)
    baseline_dice = float(
        baseline_run.data.metrics["val_dice"]
    )
else:
    baseline_dice = 0.0 # pierwsze trenowanie
```

Logika decyzyjna

Model jest uznany za ulepszony, jeśli obie metryki walidacyjne są wyższe od baseline:

- `val_dice > baseline_dice`
- `val_iou > baseline_iou`

Jeśli warunek jest spełniony, skrypt promuje nowy model do etapu Production w MLflow i archiwizuje poprzednią wersję. Wynik jest przekazywany jako output do GitHub Actions (`echo "improved=true" >> $GITHUB_OUTPUT`).

5.5 Automatyczne wdrożenie

Po scaleniu pull requesta z main uruchamiany jest workflow budujący i wdrażający nowy obraz Docker.

Budowanie obrazu

Obraz jest budowany na podstawie pliku `docker/Dockerfile.serve`. Używa warstwy bazowej z PyTorch CPU, kopiuje kod API i instaluje zależności z `requirements.txt`. Po zbudowaniu obraz jest tagowany numerem wersji i hashem commita, a następnie wysyłany do Amazon ECR:

```
docker build -f docker/Dockerfile.serve -t model-api .
docker tag model-api:latest $ECR_REGISTRY/model-api:$VERSION
docker push $ECR_REGISTRY/model-api:$VERSION
```

Wdrożenie Helm

Helm [6] aktualizuje deployment na klastrze k3s [17] nową wersją obrazu. Konfiguracja wdrożenia zdefiniowana jest w `infrastructure/helm-values.yaml`. Po wdrożeniu automatycznie wykonywany jest smoke test sprawdzający dostępność endpointów `/health`, `/predict` i `/metrics`.

6 Implementacja wdrożenia oraz użycia WIP

Rozdział opisuje infrastrukturę, na której działa system, sposób jej konfiguracji oraz scenariusze użycia z perspektywy użytkownika.

6.1 Infrastruktura Terraform

Infrastruktura AWS jest zdefiniowana jako kod przy użyciu Terraform [1]. Definicje znajdują się w katalogu `devops/terraform/`. Zastosowana architektura minimalizuje koszty — używa wyłącznie zasobów niezbędnych do działania projektu i unika kosztownych komponentów zarządzanych przez AWS (EKS, RDS, NAT Gateway, Load Balancer).

Tworzone zasoby:

- **VPC i podsieć publiczna** — prosta sieć z bezpośrednim dostępem do internetu przez Internet Gateway; eliminuje potrzebę NAT Gateway,
- **Security Group** — otwiera porty SSH (22), HTTP (80), NodePort Kubernetes (30000–32767) i port MLflow (5000) dla dozwolonych adresów IP,
- **Instancja EC2** — uruchamiana na żądanie przez workflow [5]; typ instancji konfigurowalny przez zmienną (`t3.xlarge` dla produkcji ML),
- **Amazon S3** — dwa bucket'y: jeden dla artefaktów MLflow, drugi opcjonalnie dla danych treningowych DVC,
- **Amazon ECR** — rejestr obrazów Docker dla API modelu,
- **IAM Role** — rola instancji EC2 z uprawnieniami do odczytu i zapisu S3 oraz ECR.

Infrastruktura uruchamiana jest jednorazowo poleceniem `terraform apply`. Instancja EC2 jest następnie zatrzymywana i uruchamiana automatycznie przez workflow tylko na czas treningu i wdrożenia, co istotnie obniża koszty eksploatacji.

6.2 Konfiguracja EC2 i k3s

Konfiguracja instancji EC2 odbywa się automatycznie przy pierwszym uruchomieniu za pomocą skryptu `user-data`. Skrypt jest szablonowany przez Terraform i zawiera zmienne takie jak adres bucket'a S3 czy region.

Kroki wykonywane przez `user-data`:

1. Instalacja zależności systemowych (`dnf install -y git curl libicu`).
2. Instalacja k3s — uproszczonego Kubernetes [17].
3. Instalacja MLflow i boto3 przez pip: `pip3 install -ignore-installed mlflow boto3`.
4. Konfiguracja MLflow jako usługi systemd:

```
[Service]
ExecStart=mlflow server \
  --host 0.0.0.0 --port 5000 \
  --default-artifact-root s3://${mlflow_bucket}/
```

5. Instalacja GitHub Actions runnera jako usługi systemd.

Instancja korzysta z Amazon Linux 2023, który oferuje nowszy OpenSSL 3.0 i lepszy ekosystem pakietów niż poprzednia wersja Amazon Linux 2. Po zakończeniu `user-data` instancja jest gotowa do przyjęcia zadań z GitHub Actions bez dalszej konfiguracji ręcznej.

6.3 Konfiguracja Helm

API modelu wdrażane jest na klaster k3s za pomocą Helm [6]. Chart konfiguruje następujące zasoby Kubernetes:

- **Deployment** — jedna replika kontenera z API; konfiguruje żądania i limity CPU/RAM,
- **Service (NodePort)** — udostępnia API na porcie 30080 węzła EC2; eliminuje potrzebę Load Balancera,
- **ConfigMap** — zmienne środowiskowe aplikacji (URI MLflow, ścieżki do modelu),
- **Secret** — poświadczenia AWS do pobrania modelu z S3.

Kluczowe wartości w pliku `infrastructure/helm-values.yaml`:

```
image:
  repository: <account>.dkr.ecr.us-east-1.amazonaws.com/model-api
  tag: latest
env:
  MLFLOW_TRACKING_URI: http://10.0.1.x:5000
  MODEL_STAGE: Production
resources:
  requests:
    cpu: "500m"
    memory: "1Gi"
```

Po wdrożeniu API jest dostępne pod adresem `http://<ec2-public-ip>:30080`. Dostępne endpointy to `/health` (status usługi), `/predict` (predykcja segmentacji) i `/metrics` (metryki Prometheus).

6.4 Stos monitorowania

Monitoring oparty na Prometheus [16] i Grafana wdrażany jest jako oddzielny stos na tym samym klastrze k3s. Składa się z trzech komponentów:

- **Prometheus** — zbiera metryki z endpointu `/metrics` API modelu co 15 sekund; konfiguracja scraping w `prometheus.yml`,
- **Grafana** — wizualizuje metryki na predefiniowanych dashboardach; dostępna na porcie NodePort 30300,
- **kube-state-metrics** — dostarcza metryki Kubernetes (stan podów, zużycie zasobów węzła).

API modelu eksponuje metryki w formacie Prometheus przy użyciu biblioteki `prometheus_client`. Śledzone metryki to:

- `prediction_requests_total` — łączna liczba żądań predykcji,

- `prediction_duration_seconds` — histogram czasów odpowiedzi,
- `prediction_errors_total` — liczba błędów.

6.5 Workflow użytkownika

6.5.1 Scenariusz 1: Dodanie nowych danych treningowych

Jest to główny scenariusz użycia systemu. Użytkownik chcący dostarczyć nowe obrazy wodomierzy wykonuje następujące kroki:

1. Umieszcza pliki obrazów w `WMS/data/training/images/` i odpowiadające im maski w `WMS/data/training/masks/`.
2. Dodaje pliki do staging area Git: `git add WMS/data/training/`.
3. Tworzy commit: `git commit -m "data: add N new training samples"`.
4. Wywołuje `git push`.

Hook pre-push automatycznie przekierowuje push na gałąź `data/TIMESTAMP`. Dalej system działa bez udziału użytkownika: walidacja, trening, bramka jakości, ewentualne scalenie. Użytkownik może śledzić postęp w zakładce Actions repozytorium GitHub.

6.5.2 Scenariusz 2: Predykcja lokalna

Użytkownik może uruchomić predykcję na nowych obrazach bez uruchamiania infrastruktury chmurowej:

1. Przy pierwszym użyciu pobiera aktualny model Production:

```
python WMS/scripts/sync_model_aws.py
```

2. Umieszcza obrazy do segmentacji w `WMS/data/predict/input/`.
3. Uruchamia predykcję: `python WMS/predicts.py`
4. Wyniki zapisywane są w `WMS/data/predict/output/`.

Skrypt `sync_model_aws.py` pobiera model zarejestrowany w MLflow jako Production i zapisuje go lokalnie. Model jest przechowywany w katalogu ignorowanym przez Git, więc nie trafia do repozytorium.

6.5.3 Scenariusz 3: Ręczne uruchomienie treningu

W sytuacjach wyjątkowych (np. zmiana hiperparametrów bez nowych danych) trening można uruchomić ręcznie przez zakładkę *Actions* w GitHub, wybierając workflow `train.yml` i klikając *Run workflow*. Ten tryb wymaga podania parametrów: branch'u z danymi oraz opcjonalnie liczby epok. Wynik treningu jest rejestrowany w MLflow, ale nie tworzy automatycznego pull requesta — decyzja o wdrożeniu należy do użytkownika.

7 Eksperymenty i analiza wyników WIP

7.1 Metodologia porównania

Celem eksperymentów jest ilościowe i jakościowe porównanie trzech podejść do zarządzania cyklem życia modelu uczenia maszynowego [10]:

1. **Podejście ręczne** — wszystkie kroki (przygotowanie danych, trening, walidacja, wdrożenie) wykonywane przez człowieka bez automatyzacji.
2. **Uproszczone CI/CD** — pipeline z częściową automatyzacją (automatyczny trening po merge, ręczna walidacja jakości i wdrożenie).
3. **Zautomatyzowane CI/CD (niniejszy projekt)** — pełna automatyzacja od push danych do wdrożenia modelu.

Porównanie obejmuje następujące wymiary:

- czas potrzebny do wykonania każdego scenariusza,
- jakość modelu (metryki Dice i IoU),
- pracochłonność (liczba ręcznych kroków),
- ryzyko błędu ludzkiego,
- skalowalność procesu.

7.2 Czasy wykonania scenariuszy

Tabela 1 przedstawia zmierzone czasy dla każdego podejścia w scenariuszu dodania nowej partii danych treningowych i wdrożenia ulepszanego modelu.

Tabela 1 Porównanie czasów wykonania scenariuszy

Etap	Ręczny	Uproszczone CI/CD	Zautomatyzowane CI/CD
Przygotowanie danych	[uzup.]	[uzup.]	< 1 min
Walidacja danych	[uzup.]	[uzup.]	[uzup.]
Trening modelu	[uzup.]	[uzup.]	[uzup.]
Ocena jakości	[uzup.]	[uzup.]	< 1 min
Wdrożenie	[uzup.]	[uzup.]	[uzup.]
Łącznie	[uzup.]	[uzup.]	[uzup.]

7.3 Metryki modelu

Tabela 2 przedstawia metryki modelu dla kolejnych wersji wytrenowanych w ramach eksperymentów. Baseline pochodzi z repozytorium referencyjnego *Water-Meters-Segmentation* [9]. Metryki są rejestrowane automatycznie przez MLflow [18] podczas każdego treningu.

Tabela 2 Metryki jakości modelu dla kolejnych wersji

Wersja modelu	Dice	IoU	Uwagi
Baseline (referencyjny)	0,9275	0,8865	Oryginalny model
v1 — pierwsze trenowanie	[uzup.]	[uzup.]	[uzup.]
v2 — po dodaniu danych	[uzup.]	[uzup.]	[uzup.]

7.4 Metryki infrastruktury

7.4.1 Zużycie zasobów podczas treningu

Podczas treningu modelu monitorowano obciążenie instancji EC2. Kluczowe obserwacje:

- szczytowe zużycie CPU:
- szczytowe zużycie RAM:
- czas treningu jednej epoki:

7.4.2 Zużycie zasobów podczas serwowania

API modelu serwującego predykcje charakteryzuje się następującymi parametrami wydajnościowymi:

- średni czas odpowiedzi dla jednego obrazu:
- przepustowość:
- zużycie RAM przez pod:

7.4.3 Koszty infrastruktury AWS

Tabela 3 Szacowane koszty infrastruktury AWS

Zasób	Koszt	Uwagi
EC2 t3.large (trening)	[uzup.] USD/h	Uruchamiana tylko podczas treningu
EC2 t3.large (serwowanie)	[uzup.] USD/h	Stop/start przez workflow
S3 (artefakty)	[uzup.] USD/mies.	Zależy od liczby modeli
ECR (obrazy)	[uzup.] USD/mies.	
Łącznie (projekt)	[uzup.] USD	Za cały okres realizacji

7.5 Ocena jakościowa

7.5.1 Łatwość wdrożenia

7.5.2 Stabilność systemu

W trakcie testów zaobserwowano następujące zachowania systemu w sytuacjach wyjątkowych:

- **Niepoprawne dane** — walidator odrzucał dane z komunikatem i nie uruchamiał treningu,
- **Model gorszy od baseline** — bramka jakości odrzucała model bez tworzenia PR,
- **Awaria instancji EC2** — job stop-infra uruchamiał się zawsze (if: always()), zapobiegając wyciekom kosztów.

7.5.3 Skalowalność

Zastosowane podejście ma następujące ograniczenia skalowalności:

- single-node k3s — brak wysokiej dostępności,
- jeden runner EC2 — brak równoległych treningów,
- model proof of concept — nie testowano przy zbiorach > 1000 obrazów.

8 Podsumowanie i wnioski WIP

8.1 Realizacja celów pracy

Niniejsza praca stawiała sobie za cel zaprojektowanie, implementację i ocenę zautomatyzowanego pipeline CI/CD dla modeli uczenia maszynowego. Poniżej zestawiono cele sformułowane we wprowadzeniu z oceną stopnia ich realizacji.

- **Zaprojektowanie architektury systemu automatyzacji** —
- **Implementacja pipeline CI/CD integrującego walidację danych, trening i wdrożenie** —
- **Wprowadzenie obiektywnej bramki jakości opartej na metrykach ML** —
- **Porównanie podejścia ręcznego z zautomatyzowanym** —
- **Realizacja w ramach budżetu AWS Academy (~\$50)** —

8.2 Wnioski

Na podstawie przeprowadzonych eksperymentów i obserwacji sformułowano następujące wnioski. Kontekst porównawczy oparty jest na literaturze z zakresu DevOps [7] oraz MLOps [10].

1. Automatyzacja pipeline CI/CD dla modeli ML istotnie skraca czas potrzebny do wdrożenia nowej wersji modelu w porównaniu do podejścia ręcznego.
2. Wprowadzenie dynamicznej bramki jakości opartej na metrykach MLflow eliminuje ryzyko przypadkowego wdrożenia modelu gorszego od aktualnie produkcyjnego, co jest szczególnie istotne przy niedeterministycznym procesie trenowania.
3. Wersjonowanie danych za pomocą DVC [8] zapewnia pełną odtwarzalność eksperymentów — dla dowolnej wersji modelu możliwe jest odtworzenie dokładnego zbioru danych, na którym był trenowany.
4. Zastosowanie k3s zamiast zarządzanego Kubernetes (EKS) okazało się wystarczające dla projektu proof of concept i pozwoliło znacząco ograniczyć koszty infrastruktury.
5. Architektura stop/start instancji EC2 jest efektywna kosztowo, ale wprowadza opóźnienie na uruchomienie instancji i runnera (~3–5 minut). W środowisku produkcyjnym z wymaganiami SLA należałoby rozważyć utrzymywanie instancji aktywnej.

8.3 Ograniczenia

Projekt realizowany był jako proof of concept z następującymi świadomymi ograniczeniami:

- **Budżet AWS Academy** — sesje laboratoryjne wygasają co 4 godziny, IAM role mają ograniczone uprawnienia (m.in. brak `s3:PutObject` dla lokalnych poświadczeń). Wymusiło to przeniesienie wszystkich operacji S3 do GitHub Actions.
- **Single-node k3s** — brak wysokiej dostępności. Awaria instancji EC2 powoduje niedostępność całego systemu.
- **Mały zbiór danych** — eksperyment przeprowadzono na zbiorze
- **Jeden model** — architektura jest zaprojektowana pod konkretny model (U-Net dla segmentacji). Generalizacja na inne typy modeli wymagałaby modyfikacji skryptów walidacji

i bramki jakości.

8.4 Kierunki dalszego rozwoju

System stanowi solidną bazę, którą można rozbudować w kilku kierunkach:

- **Multi-node Kubernetes** — zastąpienie single-node k3s klastrem zapewniającym wysoką dostępność i możliwość skalowania poziomego.
- **Experiment tracking** — rozbudowa wykorzystania MLflow o śledzenie hiperparametrów i automatyczne porównywanie wielu eksperymentów.
- **Data drift monitoring** — dodanie mechanizmu wykrywającego rozbieżność między danymi treningowymi a produkcyjnymi.
- **A/B testing** — wdrożenie mechanizmu stopniowego rollout nowej wersji modelu z automatycznym rollback przy pogorszeniu metryk.
- **Generalizacja** — abstrakcja pipeline pod inne typy modeli i zadań ML, nie tylko segmentację obrazów.

Literatura

- [1] Yevgeniy Brikman. *Terraform: Up and Running*. O'Reilly Media, 3 edition, 2022. ISBN 978-1-098-11674-3. <https://www.oreilly.com/library/view/terraform-up-and/9781098116736/>.
- [2] Brendan Burns, Brian Grant, David Oppenheimer, Eric Brewer, and John Wilkes. Borg, omega, and kubernetes. In *ACM Queue*, volume 14, pages 70–93, 2016. doi: 10.1145/2898442.2898444. <https://dl.acm.org/doi/10.1145/2898442.2898444>.
- [3] Scott Chacon and Ben Straub. *Pro Git*. Apress, 2 edition, 2014. Dostęp: 10.02.2026.
- [4] DataFight. U-net i segmentacja obrazu. <https://datafight.wordpress.com/2020/05/04/u-net-i-segmentacja-obrazu/>, 2020. Dostęp: 10.02.2026.
- [5] GitHub Inc. Github actions documentation. <https://docs.github.com/en/actions>, 2024. Dostęp: 10.02.2026.
- [6] Helm Authors. Helm — the kubernetes package manager. <https://helm.sh/docs/>, 2023. Dostęp: 10.02.2026.
- [7] Jez Humble and David Farley. *Continuous Delivery: Reliable Software Releases through Build, Test, and Deployment Automation*. Addison-Wesley, 2010. ISBN 978-0-321-60191-9. <https://www.informit.com/store/continuous-delivery-9780321601919>.
- [8] Iterative.ai. Data version control — open-source version control system for machine learning projects. <https://dvc.org/doc>, 2023. Dostęp: 10.02.2026.
- [9] Kaggle. Yandex toloka water meters dataset. <https://www.kaggle.com/datasets/tapakah68/yandextoloka-water-meters-dataset>, 2023. Dostęp: 10.02.2026.
- [10] Dominik Kreuzberger, Niklas Kühl, and Sebastian Hirschl. Machine learning operations (MLOps): Overview, definition, and architecture. *IEEE Access*, 11:31866–31879, 2023. doi: 10.1109/ACCESS.2023.3262138.
- [11] Grafana Labs. Grafana documentation: Introduction. <https://grafana.com/docs/grafana/latest/fundamentals/>, 2024. Dostęp: 16.02.2026.
- [12] Mike Loukides. What is devops? <https://cdn.bookey.app/files/pdf/book/en/what-is-devops-.pdf>, 2012. Bookey summary PDF, accessed: 2026-02-16.
- [13] Dirk Merkel. Docker: Lightweight linux containers for consistent development and deployment. *Linux Journal*, 2014(239), 2014. <https://www.linuxjournal.com/content/docker-lightweight-linux-containers-consistent-development-and-deployment>.
- [14] Nehal Navlani. Ci/cd pipeline - system design. <https://www.geeksforgeeks.org/system-design/cicd-pipeline-system-design/>, 2025.

- [15] MLflow Project. MLflow model registry. <https://mlflow.org/docs/latest/ml/model-registry/>, 2024. Dostęp: 16.02.2026.
- [16] Prometheus Authors. Prometheus — monitoring system and time series database. <https://prometheus.io/docs/introduction/overview/>, 2023. Dostęp: 10.02.2026.
- [17] Rancher Labs. k3s — lightweight kubernetes. <https://docs.k3s.io>, 2023. Dostęp: 10.02.2026.
- [18] Matei Zaharia, Andrew Chen, Aaron Davidson, Ali Ghodsi, Sue Ann Hong, Andy Murching, Tomas Nykodym, Paul Ogilvie, Mani Parkhe, Aarth Singh, et al. MLflow: A system for machine-learning lifecycle management. In *Proceedings of the Workshop on Data Management for End-to-End Machine Learning (DEEM)*. ACM, 2018. doi: 10.1145/3399579.3399604.